

Coesão e princípios SOLID

Sistema de hackathons em Python • Revisão em 9 de setembro de 2026

Definição adotada: $LCOM(C)$ é o número de pares de métodos da classe C que não usam atributos em comum. Portanto, $LCOM(C) = P$. Não se subtrai Q . Quanto menor o valor, menor a falta de coesão indicada por esse critério.

Critério de cálculo

Análise estática somente das classes de `models.py`. Contam-se acessos diretos a atributos de instância, incluindo `db`. Métodos declarados entram no cálculo; `__init__`, métodos gerados por `dataclass`, funções livres e métodos herdados não entram na contagem. Atributos herdados são reconhecidos; `PAPEL` é constante de classe e não entra. Chamadas a outros métodos não propagam atributos no resultado principal.

Classe	Métodos	Pares totais	$LCOM = P$
Organizador	6	15	9
Hackathon	3	3	0
Usuario	2	1	0
Participante	4	6	0
Equipe	10	45	17
Projeto	0	0	0
Mentor	4	6	5
Mentoria	0	0	0
Jurado	4	6	5
Avaliacao	0	0	0

Projeto, Mentoria e Avaliacao têm zero pares por não declarar métodos. Seu resultado aritmético é zero, mas não demonstra alta coesão comportamental.

Exemplo e interpretação

Mentor possui 4 métodos e $4 \times 3 / 2 = 6$ pares. Apenas `visualizar_mentorias` e `salvar_mentoria` compartilham atributos diretamente. Os outros 5 pares têm interseção vazia: **$LCOM(Mentor) = 5$** .

Três das sete classes com métodos têm $LCOM$ zero: `Hackathon`, `Usuario` e `Participante`. `Equipe` apresenta o maior valor (17), mas o número absoluto depende da quantidade de métodos. Delegações internas podem aumentar P ; compartilhar apenas o banco pode reduzi-lo. Logo, a métrica orienta a análise e não prova, sozinha, qualidade ou necessidade de refatoração.

Justificativa de três princípios SOLID

Os princípios abaixo têm evidências no código nos escopos descritos. Não se afirma conformidade integral de toda a arquitetura.

SRP Responsabilidade única

Evidência: Mentor reúne operações de mentoria, Jurado reúne avaliações e Participante reúne ações de participação em equipes. Hackathon.ranking calcula a classificação das equipes do evento.

Justificativa: regras específicas de mentoria podem mudar sem alterar os métodos de avaliação de Jurado. As responsabilidades de domínio estão identificadas por papel. **Limitação:** ainda existe SQL nos modelos, portanto a separação entre regras e persistência é parcial.

ISP Segregação de interfaces

Evidência: Usuario oferece validar e sair_hackathon. Participante, Mentor e Jurado acrescentam somente operações do próprio papel. Um mentor não herda entrar_equipe nem avaliar_projeto.

Justificativa: cada menu usa a API do papel correspondente, sem exigir métodos irrelevantes ou implementações vazias nos outros papéis. A separação ocorre pelas classes, mesmo sem interfaces formais em Python.

LSP Substituição de Liskov

Evidência: Participante, Mentor e Jurado herdam validar e sair_hackathon sem sobrescrita. Esses métodos podem ser chamados uniformemente em qualquer um dos três papéis válidos.

Justificativa: os objetos válidos preservam o contrato comum de verificar o vínculo, resolver eventual liderança e sair sem apagar o histórico. **Limitação:** Usuario é uma base implícita, sem PAPEL definido nem declaração abstrata; a conclusão vale para os subtipos construídos pelo sistema, não para instanciar Usuario diretamente.

Conclusão

Há evidências de SRP, ISP e LSP nos pontos indicados. Para buscar menor LCOM, convém manter ações relacionadas na mesma classe e revisar grupos independentes, sem duplicar código apenas para reduzir P. Este relatório não modifica o sistema.

Reprodução e referências

Execute `python calcular_lcom.py caminho/para/hackathon`. O arquivo `resultados_lcom.json` registra os valores, atributos por método e hashes dos arquivos medidos. Use o campo direto como resultado principal; os demais cenários são complementares.

Fundamentação: Robert C. Martin, [The Single Responsibility Principle](#) (2014) e [Design Principles and Design Patterns](#) (2000). A definição $LCOM = P$ segue o critério solicitado para esta atividade.